IB113 Úvod do programování a algoritmizace

Cvičení 2

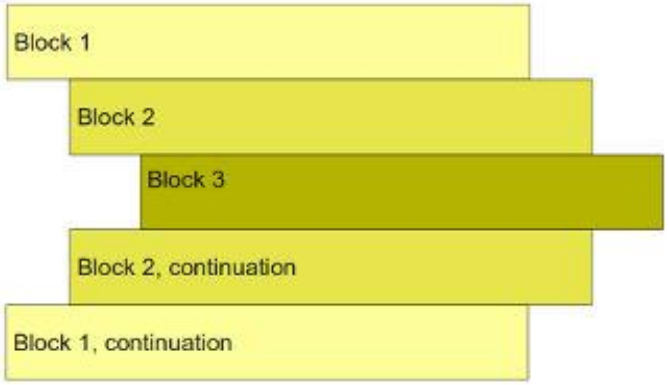
Jaromír Plhák





Syntaxe Pythonu (1)

- Case sensitive (ne cASe INSensiTIvE)
- Konec příkazu: konec řádku (ne ;)
- Řádkový komentář: #
- Blokový komentář: """
- Bloky kódu: odsazením (ne {} ani begin. . . end)




```
from math import sqrt
n = input("Maximal Number? ")
n = int(n)+1
for a in range(1,n):
    for b in range(a,n):
        c_square = a**2 + b**2
        c = int(sqrt(c_square))
        if ((c_square - c**2) == 0):
            print(a, b, c)
```

Syntaxe Pythonu (2)

- Méně znaků, přirozená čitelnost
  - Není nutné používat všude závorky
- Dobrý zvyk: po dvojtečce nový řádek a 4 mezery
  - Nastavení znaku Tab v IDE

Algoritmus a program

- Algoritmus: konečná postupnost přesně definovaných instrukcí pro splnění určité úlohy
  - [MCCONNELL, Steve: *Code Complete: A Practical Handbook of Software Construction*. První vydání. Microsoft Press, 1993. 857 stran. ISBN 1-55615-484-4.]
  - Reprezentace: slovně, graficky, pseudokódem
  - Např.: Vypiš na obrazovku číslo jedna!
- Program: přepis algoritmu do programovacího jazyka
  - Reprezentace: soubor s kódem
  - Např.: print("1")
- Cíl předmětu: vymýšlet algoritmy pro řešení problémů a zapisovat algoritmy ve formě programů

Proměnné

- Místo v paměti, kde je uložená hodnota pod nějakým jménem
  - Smysluplné a výstižné pojmenování[1], ne my_super_variable
  - Obvykle malé písmena, čísla a podtržítka mezi slovy
  - Ne rezervovaná slova
  - Používejte angličtinu pro pojmenovávání proměnných
- Přiřazovací příkaz: proměnná = hodnota
- Porovnání: proměnná == hodnota

- [1] http://www.hovnokod.cz/450

Datový typ

- Množina hodnot spolu s operacemi na těchto hodnotách
- Celé číslo (int): 42, 0, -14
- Desetinné číslo (float): 42.0, 0.21, -7.59876, 3e8
- Logická hodnota (bool): True, False
- Řetězec (string): "The answer is 42!", '0.21'
- Seznam (list): [1, 4, 9, 16, 25]
- …
- Python: implicitní typovaní (typ se k proměnné nepíše)

Datový typ

- Zjištění typu: funkce type()
- **>>> type (1)**
- **<class 'int '>**
- **>>> type (" Hello ")**
- **<class 'str'>**
- Přetypování: funkce int(), float(), str()
- **>>> str (1)**
- **'1'**

Operátory

- Jazyková konstrukce podobná funkci, která umožňuje operace na hodnotách
- Jeho argument: operand
- Operátory přiřazení: =, +=, -=, *=, /=, %=, **=
- Aritmetické operátory: +, -, *, /, //, %, **
- Relační operátory: ==, !=, <, >, <=, >=
- Logické operátory: and, or, not
- Bitové operátory: <<, >>, &, |, ^, ~
- Jiné: [], ., is, is not, in, not in

Relační operátory

- Úloha: jaké hodnoty budou v proměnných a až f?

```
>>> a = 4 == 4
>>> b = 8 != 5
>>> c = 9 < 4
>>> d = 9 <= 4
>>> e = 5 > 3
>>> f = 5 >= 5
```

```
>>> a = True
>>> b = True
>>> c = False
>>> d = False
>>> e = True
>>> f = True
```

- Oddělujte operátory mezerou na obou stranách

Logické operátory

- Úloha: jaké hodnoty budou v proměnných a až i?

```
>>> a = True and True
>>> b = True and False
>>> c = False and False
>>> d = True or True
>>> e = True or False
>>> f = False or False
>>> g = not True
>>> h = not False
>>> i = not not True
```

```
>>> a = True
>>> b = False
>>> c = False
>>> d = True
>>> e = True
>>> f = False
>>> g = False
>>> h = True
>>> i = True
```

Výraz

- Kombinace konstant, proměnných, operátorů a funkcí
- Nejprve aritmetické, potom relační, nakonec logické

>>> 5 + 16 >= 5 + 4*2

>>> 5 + 16 >= 5 + 8

>>> 21 >= 5 + 8

>>> 21 >= 13

>>> True

- Při pochybnostech použijte závorky

Příkaz print

- Co se vypíše?

```
>>> a = 1
>>> b = 2
>>> print(a + b)
>>> print(a, b)

>>> a = "1"
>>> b = "2"
>>> print(a + b)
>>> print(a, b)
```

Příkaz print

- Pozor na operace s nekompatibilnými typy:

```
>>> a = 1
>>> b = "2"
>>> print(a + b)
TypeError : unsupported operand type (s) for +: 'int' and 'str'
```

- Přetypování:

```
>>> print(a + int (b))
3
```

Základní konstrukce – podmíněný příkaz

if <podmínka>:

    příkaz1

else:

    příkaz2

- Podle toho, zda platí podmínka, se provede jedna z větví
- Podmínka – typicky výraz nad proměnnými
- else větev nepovinná
- Vícenásobné větvení: if - elif - ... - else

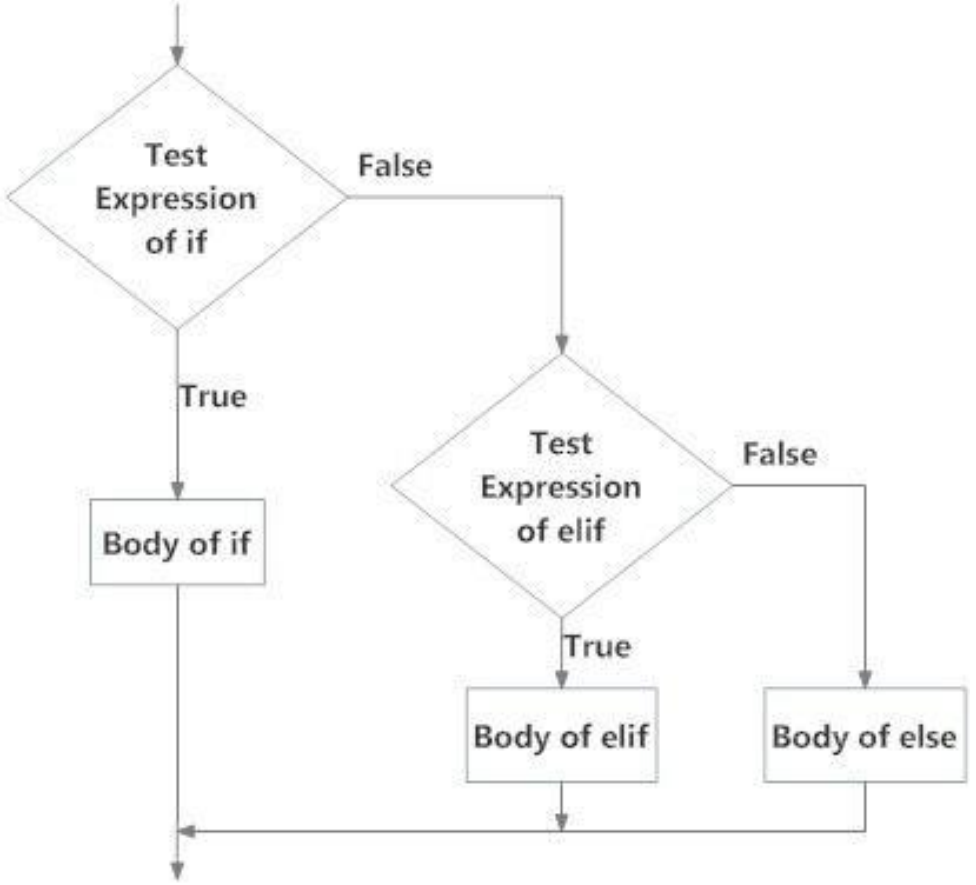


Fig: Operation of if...elif...else statement

Zdroj: http://www.programiz.com/python-programming/if-elif-else

Základní konstrukce – Ternární podmínka

- Mějme kód:

```
max = 0
if a > b:
        max = a
else :
        max = b
```

- Ekvivalentní zápis:

```
max = a if a > b else b
```

- Čteme: „Nechť max je a pokud a je větší než b, jinak nechť max je b.“

Základní konstrukce – cyklus for

- Opakované provádění sekvence příkazů
- Známý počet opakování cyklu
- Příkaz for

```
for i in range(3):
    print(i)
```

- Proměnná *i* postupně nabývá celočíselné hodnoty z rozsahu [0, 3)
- range(n) – interval od 0 do n - 1 (tj. n opakování)

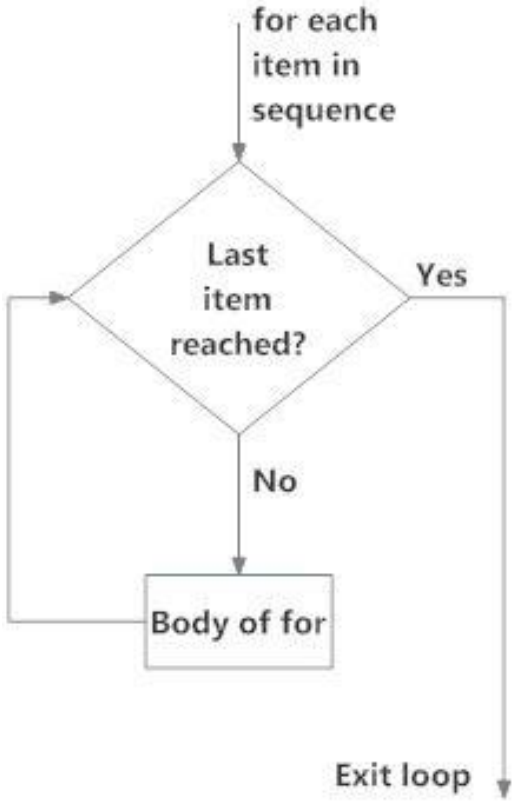


Fig: operation of for loop

Zdroj: http://www.programiz.com/python-programming/for-loop

Základní konstrukce – cyklus while

- Neznámý počet opakování cyklu
- Příkaz while
- Opakuj dokud platí podmínka

```
while <podmínka>:
    příkazy
```

```
choice = input ("Like the course ? (y/n)")

while choice != "y" and choice != "n":
    choice = input ("Sorry , I didn 't catch that . Enter again : ")
```

Procedury a funkce - motivace

- Znovupoužití kódu (s různými hodnotami):

```
def powers(base, num):
    current = 1
    for i in range(num):
        print(current, end=" ")
        current = current * base
    print()

powers(2, 10)  # volání funkce, kde base = 2 a n = 10
```

- Jako refrén v písničce (s malými obměnami)

Procedury a funkce - definice

- Klíčové slovo def
- Mezera
- Jméno funkce
- Seznam parametrů v závorkách
  - Žádné parametry - prázdné závorky
- Dvojtečka
- Nový řádek
- Odsazení příkazu funkce

```
def my_funct (par1 , par2 , ...) : # header
        statement_1 # body
        ...
        statement_n
```

Procedury vs. funkce

- Funkce: vrátí hodnotu, může být použita ve výrazu
  - Poslední příkaz musí být return hodnota
- Procedura: jen sekvence příkazů, nevrací hodnotu
  - Python: procedura je speciální případ funkce, která obsahuje implicitní příkaz return None
  - Funkce, která „vrátí nic“, prázdný objekt
- Dále budeme označovat jako **funkce**

Příklad funkce

- Funkce by měla mít smysluplné a výstižné jméno
  - Ne my_super_function()
- Je dobrým zvykem komentovat chovaní funkce
  - Tzv. *docstring*

```
def hello ():
        """ Print 'Hello !' to console . """
        print (" Hello !")
```

- Funkční volání: použití jména funkce a zadaní všech potřebných parametrů
  - Způsobí vykonaní kódu ve funkci

```
>>> hello () # No parameters
Hello !
```

Parametry funkcí

- Formální parametry: vstupy funkce v její definici

```
def power (base , exponent ):
        result = base ** exponent
        print (base, "to the power of", exponent, "is", result)
```

- Skutečné parametry (argumenty): hodnoty, které volající příkaz posílá funkci
- Formální parametry jsou při volání nahrazeny skutečnými

```
>>> power (3, 4)
# base initializes to 3, exponent to 4
3 to the power of 4 is 81.
```

Formátování kódu

- Smysluplná funkce
  - http://www.hovnokod.cz/367
- Vhodné jméno funkce
  - http://www.hovnokod.cz/392
- Vhodné komentáře (pokud jsou potřebné)
  - http://www.hovnokod.cz/401
- Krátké funkce (má jeden účel)
  - http://www.hovnokod.cz/353

Formátování kódu

- Dlouhou funkci rozdělte na menší
  - http://www.hovnokod.cz/441
- Nezanořujeme se příliš
  - http://www.hovnokod.cz/1429
- Maximálně 3 úrovně (podmínek a cyklů)
- Pokud chcete kopírovat kód, vytvořte pro něj funkci
  - http://www.hovnokod.cz/461
- Použijte vlastní odhad: pokud si při čtení kódu chcete vypíchnout oči a skočit z okna, je pravděpodobně něco v nepořádku
  - http://www.hovnokod.cz/470
  - http://www.hovnokod.cz/501

Pycodestyle (dříve pep8)

- Style Guide for Python Code
  - Oficiální doporučení
  - https://www.python.org/dev/peps/pep-0008/
- Důležité body z PEP8:
  - Zarovnávání kódu; max. délka řádku 79 znaků; dva prázdné řádky mezi funkcemi
  - Mezery kolem operátorů; žádné mezery za otevírací (před uzavírací) závorkou
    - Výjimka: žádné mezery kolem = u parametrů funkcí
  - Pojmenování věcí proměnné, funkce: lower_case_with_underscores
  - Konstanty: UPPER_CASE_WITH_UNDERSCORES
  - Třídy (později): CamelCase

Úvodní úkoly

- Vytvořte program počítající objem kvádru a objem koule
  - Výsledek vypište ve formě "Objem kvadru je roven <VÝSLEDEK> cm^3."
  - HINT: print umožňuje vytisknout více parametrů oddělených mezerou
    - print("jedna =", 1) vytiskne na obrazovku "jedna = 1"
  - Pí
    - Je možné si jej nadefinovat
    - Můžeme jej najít v matematické knihovně
      - from math import pi
      - Nyní můžete používat proměnnou pi, která obsahuje hodnotu 3.14159265358979

Úkoly

- Vypište prvních 10 mocnin čísla 2.
- Vypište prvních 15 prvků Fibonnaciho posloupnosti
- Vypište prvky posloupností, jejichž první prvky vypadají následovně:
  - a) 1 -1 2 -2 3 -3 4 -4 5 -5 … (končí číslem -10)
  - b) 1 1 2 1 2 3 1 2 3 4 … (končí číslem 10)
- Nalezněte a vypište všechny dělitele daného čísla
  - Číslo zadejte pomocí proměnné
  - Čísla vypište na jeden řádek oddělená čárkou
  - Pro 0 a záporná čísla vypište chybovou hlášku
- Převeďte předchozí příklady do funkcí (procedur), které spustíte
- Oddělte hodnoty čárkou a mezerou (za poslední hodnotou není čárka)